Class 29: MLOps Practice – Continuous Training

Session 29 of 36 – MLOps Block

Data-driven Systems Engineering

Agenda

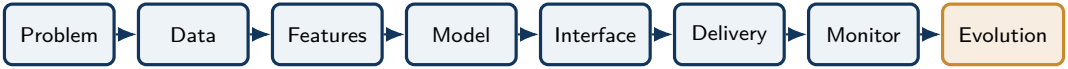
- From one-off training to continuous training
- Triggers for retraining
- Anatomy of a continuous training pipeline
- Evaluation gates and champion-challenger promotion
- Failure modes of automated retraining
- Practical Component: what exists in this repo today

Training is a pipeline, not an event

Learning goals

- Explain why manual, ad hoc retraining does not scale past a demo.
- Identify triggers that should launch a retraining pipeline: schedule, data volume, drift, and performance decay.
- Describe the stages of a continuous training pipeline and the gates between them.
- Recognize the difference between automating retraining and automating deployment of a retrained model.

Course Map and Running Cases



Today in the course

Focus: Operationalizing ML development and training as a repeatable, automatable pipeline.
Bridge: Class 28 introduced MLOps as coordination across the whole lifecycle; this class zooms into one box of that diagram, train and retrain, and asks what it takes to run it without a human kicking it off by hand every time.

Fraud case

A continuous training pipeline for fraud scoring would retrain on a schedule or when score drift is detected, then gate promotion on a held-out evaluation before replacing the serving model.

Pasteurization case

A continuous training pipeline for the pasteurization forecaster would retrain as new batches complete, gated by a check that the new model does not regress on known abnormal trajectories.

Course thread: Automation without validation just moves risk faster; a training pipeline needs the same gates

From one-off training to continuous training

- In a notebook, training is something a person runs once and inspects.
- In production, the same code needs to run again: new data arrives, labels catch up, or performance decays.
- Continuous training (CT) means retraining is a repeatable, automatable job, not a manual ritual.

What “continuous” means here

- Not continuous like a sensor stream: continuous training usually runs on a cadence, not every second.
- The point is that the pipeline can run unattended and produce a trustworthy candidate model each time.
- Frequency should match the business reason for retraining, not a fixed habit.

Triggers for retraining

Trigger	When it fires
Schedule	A fixed cadence, for example nightly or weekly.
Data volume	Enough new labeled data has accumulated to justify a refresh.
Drift signal	A monitoring alert from Class 32's detectors fires.
Performance decay	A tracked metric drops below an agreed threshold.
Manual request	A team member has a specific reason to trigger a run.

Anatomy of a continuous training pipeline



- Every arrow is a gate: a candidate that fails a stage should not move forward automatically.

Data validation before training

- Schema checks: are the expected columns present, with the expected types?
- Freshness checks: is the data as recent as the trigger assumed?
- Volume and balance checks: is there enough signal, and is the label distribution sane?
- A pipeline that trains on broken data just automates producing a broken model faster.

Evaluation gates before promotion

- Compare the new candidate against the current champion on a frozen, held-out evaluation set.
- Check a business metric, not only a statistical one: review load, false-alert rate, or missed-detection cost.
- A candidate that wins on one metric but regresses on another needs a human decision, not an automatic merge.

Champion-challenger extends into training

- The same champion-challenger idea from monitoring applies to retraining decisions.
- A newly trained candidate is a challenger until it earns promotion through the evaluation gate.
- Shadow evaluation on live traffic, before full promotion, catches problems that offline metrics miss.

Where continuous training meets CI/CD

- The retraining job itself can be triggered the same way as any other automated workflow: on a schedule or an event.
- Class 31's GitHub Actions cron trigger is a small but real building block for exactly this kind of scheduled job.
- Packaging, testing, and versioning from Class 31 apply to the training code path, not only the serving code path.

Failure modes of automated retraining

- Feedback loops: a fraud model's own blocking decisions can bias the labels it later retrains on.
- Drift masking: retraining on recent data can quietly bake in a temporary anomaly as if it were normal.
- Silent schema drift: an upstream field changes meaning without anyone updating the training code.
- Unbounded automation: nothing stops a bad candidate from being promoted if a gate is missing or too weak.

Rule

Every automated stage needs an explicit condition for stopping, not just for proceeding.

Governance for an automated retraining path

- Log every retraining run: trigger reason, data snapshot, metrics, and the promote-or-reject decision.
- Keep a human approval step for high-stakes promotions, even if triggering is automatic.
- Version training code and data references together with the resulting model artifact.

Practical Component

Honest gap in this repository

There is currently no automated retraining pipeline in this repository: no scheduled job that pulls new data, retrains, evaluates, and promotes a model.

- The building blocks exist separately: training code in the notebooks and ‘streamlit/modeling.py‘, and a scheduled-trigger pattern in ‘tutorials/GitHubActions/tutorial.md‘.
- Wiring them into a real continuous training pipeline is suggested as a separate project, for example a dedicated GitHub submodule, that the course will build going forward.
- This class stays conceptual on purpose: understand the stages and gates before wiring any automation.

Repo materials we can build on

Repo activity

- ‘streamlit/modeling.py‘ already contains the training and artifact-saving logic a CT pipeline would call.
- ‘tutorials/GitHubActions/tutorial.md‘’s cron-triggered workflow is the closest existing building block for a scheduled retraining job.
- Neither piece is wired to the other yet; that gap is exactly what a continuous training project would close.

Discussion prompt

Question

If a retraining pipeline ran fully unattended, which single gate would you refuse to automate away, and why?

- This forces a concrete stance on where human judgment stays load-bearing.
- It also previews the governance theme that returns in later monitoring and scaling discussions.

Papers and materials

Papers and materials

Chip Huyen, Designing Machine Learning Systems; Google Cloud MLOps guidance on CI/CD/CT for ML; Sculley et al., Hidden Technical Debt in Machine Learning Systems; Practical MLOps.

From This Class to the Next

What stays with us

- Continuous training turns retraining from a manual ritual into a governed, repeatable pipeline.
- Triggers, data validation, evaluation gates, and promotion decisions all need explicit rules.
- Automating a bad process just produces bad models faster; the gates matter more than the automation.

Project use

Teams should now be able to describe, even without building it, what a retraining pipeline for their own project would trigger on and what would stop a bad candidate.

Next connection

Class 31 returns to the delivery side of the pipeline: Docker and GitHub Actions for packaging and shipping what Class 28 and Class 29 have described.

Course thread: Training and delivery are two automatable paths that both need the same discipline: gates, versioning, and observability.